# TODOS LOS EJERCICIOS DEL RAR

Manual de estudio · Java Swing · Código sin initComponents() ni main()

Esta versión conserva las explicaciones y el código de todos los archivos Java del paquete. Se ha eliminado el contenido completo de los métodos initComponents() y main() para que puedas centrarte en la lógica, los eventos, los atributos y los métodos propios de cada ejercicio.

Nota: En los lugares donde se ha retirado uno de esos métodos aparece un comentario indicando la omisión. En NetBeans, initComponents() suele ser código generado por el diseñador visual; no necesitas copiarlo para estudiar la lógica.

TODOS LOS EJERCICIOS DEL RAR Manual de estudio · Java Swing · Código sin initComponents() ni main() Esta versión conserva las explicaciones y el código de todos los archivos Java del paquete. Se ha eliminado el contenido completo de los métodos initComponents() y main() para que puedas centrarte en la lógica, los eventos, los atributos y los métodos propios de cada ejercicio. Nota: En los lugares donde se ha retirado uno de esos métodos aparece un comentario indicando la omisión. En NetBeans, initComponents() suele ser código generado por el diseñador visual; no necesitas copiarlo para estudiar la lógica.

TODOS LOS EJERCICIOS DEL RAR Manual de estudio con explicación por archivo y código Java completo Este documento reúne los 44 archivos .java encontrados en EJERCICIOS-DI-JAVA.rar. Cada ficha identifica la función de la clase, los componentes Swing que utiliza, los atributos y métodos detectados, los eventos principales y el código fuente del archivo. Incluye: prácticas 1.1, 1.2, 1.3, 1.4, 1.5 y los siete ejercicios extra. Se conserva el código de cada archivo para que puedas consultarlo y compararlo con NetBeans. Cómo aprovechar el manual 1. Lee primero la explicación de la práctica. 2. Identifica la clase de datos y la ventana. 3. Revisa los componentes Swing y los métodos de evento. 4. Lee el código fuente incluido. 5. Abre el archivo original en NetBeans y sigue las instrucciones de la guía práctica anterior para diseñar, ejecutar y depurar. Importante: el código generado automáticamente por el GUI Builder puede incluir initComponents() y bloques Generated Code. No los edites manualmente desde Source; cambia el diseño desde la pestaña Design de NetBeans. Índice de ejercicios y archivos practica1.1 — Primeras ventanas Swing: calculadora, acceso y saludo. Se practican JFrame, etiquetas, campos de texto, botones y eventos. VentanaCalculadora.java VentanaLogin.java VentanaSaludo.java practica1.2/apartado01 — Clase Libro y ventana de formulario: atributos, constructor, getters/setters y lectura de controles. Libro.java VentanaLibro.java practica1.2/apartado02 — SumadorPares y su ventana: separar la lógica de cálculo de la interfaz. SumadorPares.java VentanaSumador.java practica1.2/apartado03 — Personaje y ventana Warcraft: objetos con estado y comportamiento. Personaje.java VentanaWarcraft.java practica1.3/apartado01 — Catálogo de libros: reutilizar una clase de datos y mostrar varios registros. Libro.java VentanaCatálogo.java practica1.3/apartado03 — Combate con personajes especializados: herencia, clases abstractas y polimorfismo. Arquero.java Guerrero.java Jefe.java Mago.java Personaje.java VentanaCombate.java VentanaSeleccion.java practica1.4 — Juegos y ventana principal: tablero, eventos, turnos, puntuación y navegación. Jugador.java VentanaBuscaminas.java VentanaCazatesoros.java VentanaPrincipal.java practica1.5/apartado01 — Perfil con ventana secundaria: JDialog, validación y comunicación con JFrame. DialogPerfil.java VentanaPrincipal.java practica1.5/apartado02 — Catálogo de coches: objetos, JTable y modelo personalizado. Coche.java Modelo.java VentanaCatalogo.java practica1.5/apartado03 — Agenda de contactos: diálogo de alta, validación y tabla. Contacto.java DialogContacto.java Modelo.java VentanaAgenda.java practica1.5/apartado04 — Reclutamiento de astronautas: diálogo, objetos y modelo de tabla. Astronauta.java DialogReclutamiento.java Modelo.java VentanaNave.java practicaExtra/apartado01 — Registro de productos: formularios, menús y selección de categorías. Producto.java VentanaPrincipal.java VentanaRegistro.java practicaExtra/apartado02 — Tablero por turnos: botones, estado y eventos. VentanaTablero.java practicaExtra/apartado03 — Agenda con tabla: añadir y eliminar registros. VentanaAgenda.java practicaExtra/apartado04 — Registro de usuarios: contraseña, curso y aceptación de condiciones. VentanaUsuario.java practicaExtra/apartado05 — Calculadora de propina: JSlider, porcentajes y actualización dinámica. VentanaPropina.java practicaExtra/apartado06 — Teclado PIN: botones numéricos, borrar y comprobación. VentanaPin.java practicaExtra/apartado07 — Lista de tareas: JList, DefaultListModel y menú. VentanaTareas.java

# PRACTICA1.1

Primeras ventanas Swing: calculadora, acceso y saludo. Se practican JFrame, etiquetas, campos de texto, botones y eventos.

## VentanaCalculadora.java — clase VentanaCalculadora

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JFrame, JLabel, JTextField Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); jTextField1ActionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed, jTextField1ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, convierte texto a entero, actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
/**
 *
 * @author alumno
 */
public class VentanaCalculadora extends javax.swing.JFrame {
    /**
     * Creates new form VentanaCalculadora
     */
    public VentanaCalculadora() {
        initComponents();
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jTextField1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jTextField1ActionPerformed
        // TODO add your handling code here:
    }//GEN-LAST:event_jTextField1ActionPerformed
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        try {
            int num1 = Integer.parseInt(jTextField1.getText().trim());
            int num2 = Integer.parseInt(jTextField2.getText().trim());
            int suma = num1 + num2;
            jLabel3.setText(String.valueOf(suma));
        } catch (NumberFormatException e) {
            jLabel3.setText("Solo numeros");
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JLabel jLabel4;
    private javax.swing.JLabel jLabel5;
    private javax.swing.JTextField jTextField1;
    private javax.swing.JTextField jTextField2;
    // End of variables declaration//GEN-END:variables
}
```

## VentanaLogin.java — clase VentanaLogin

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JFrame, JLabel, JPasswordField, JTextField Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, lee un campo de contraseña, actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JPasswordField: campos de contraseña; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
/**
 *
 * @author alumno
 */
public class VentanaLogin extends javax.swing.JFrame {
    /**
     * Creates new form VentanaLogin
     */
    public VentanaLogin() {
        initComponents();
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        String usuario = jTextField1.getText().trim();
        String clave = new String(jPasswordField1.getPassword());
        if (usuario.equals("admin") && clave.equals("1234")) {
            jLabel3.setText("Credenciales correctas");
        } else {
            jLabel3.setText("Error. Credenciales no válidas");
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JPasswordField jPasswordField1;
    private javax.swing.JTextField jTextField1;
    // End of variables declaration//GEN-END:variables
}
```

## VentanaSaludo.java — clase VentanaSaludo

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JFrame, JLabel, JTextField Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); jTextField1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed, jTextField1ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

### Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
/**
 *
 * @author alumno
 */
public class VentanaSaludo extends javax.swing.JFrame {
    /**
     * Creates new form VentanaSaludo
     */
    public VentanaSaludo() {
        initComponents();
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        String nombre = jTextField1.getText().trim();
        if (nombre.isEmpty() || nombre.equals("Nombre")) {
            jLabel1.setText("Por favor, escribe tu nombre");
        } else {
            jLabel1.setText("Hola de nuevo, " + nombre);
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    private void jTextField1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jTextField1ActionPerformed
        // TODO add your handling code here:
    }//GEN-LAST:event_jTextField1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JTextField jTextField1;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICA1.2 · APARTADO01

Clase Libro y ventana de formulario: atributos, constructor, getters/setters y lectura de controles.

### Libro.java — clase Libro

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Atributos detectados: private int numPaginas; private String titulo; private boolean almacenado Métodos detectados: getNumPaginas(); setNumPaginas(int numPaginas); getTitulo(); setTitulo(String titulo); isAlmacenado(); setAlmacenado(boolean almacenado); toString() Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

## Código fuente completo del archivo:

```
package interfaz;
/**
 *
 * @author alumno
 */
public class Libro {
    private int numPaginas;
    private String titulo;
    private boolean almacenado;
    public Libro(int numPaginas, String titulo, boolean almacenado) {
        this.numPaginas = numPaginas;
        this.titulo = titulo;
        this.almacenado = almacenado;
    }
    public Libro() {
        numPaginas = 0;
        titulo = "";
        almacenado = false;
    }
    public int getNumPaginas() {
        return numPaginas;
    }
    public void setNumPaginas(int numPaginas) {
        this.numPaginas = numPaginas;
    }
    public String getTitulo() {
        return titulo;
    }
    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }
    public boolean isAlmacenado() {
        return almacenado;
    }
    public void setAlmacenado(boolean almacenado) {
        this.almacenado = almacenado;
    }
    @Override
    public String toString() {
        //Se pone el atributo y detras el simbolo ? y luego separado por dos puntos sale cuando se
        ejecuta el checkBox
        return titulo + " de " + numPaginas + " paginas, " + (almacenado ? "esta" : "no esta") + " en la
        biblioteca";
    }
}
```

## VentanaLibro.java — clase VentanaLibro

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JCheckBox, JFrame, JLabel, JTextField Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); jCheckBox1ActionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed, jCheckBox1ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, convierte texto a entero, consulta una casilla u opción, actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JCheckBox: casilla de verificación; JLabel: etiquetas para títulos e instrucciones; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

### Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
/**
 *
 * @author alumno
 */
public class VentanaLibro extends javax.swing.JFrame {
    /**
     * Creates new form VentanaLibro
     */
    public VentanaLibro() {
        initComponents();
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jCheckBox1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jCheckBox1ActionPerformed
        // TODO add your handling code here:
    }//GEN-LAST:event_jCheckBox1ActionPerformed
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        try {
            String titulo = jTextField2.getText().trim();
            int numPag = Integer.parseInt(jTextField1.getText().trim());
            boolean encontrado = jCheckBox1.isSelected();
            if (titulo.isEmpty()) {
                Result.setText("Por favor, escribe un titulo");
            } else if (numPag <= 0) {
                Result.setText("Debe de ser un numero positivo");
            } else {
                Libro libro = new Libro(numPag, titulo, encontrado);
                Result.setText(libro.toString());
            }
        } catch (NumberFormatException e) {
            Result.setText("El numero de paginas debe ser un numero entero");
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JLabel Result;
    private javax.swing.JButton jButton1;
    private javax.swing.JCheckBox jCheckBox1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JTextField jTextField1;
    private javax.swing.JTextField jTextField2;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICA1.2 · APARTADO02

SumadorPares y su ventana: separar la lógica de cálculo de la interfaz.

### SumadorPares.java — clase SumadorPares

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Atributos detectados: private int num1; private int num2 Métodos detectados: getNum1(); setNum1(int num1); getNum2(); setNum2(int num2); sumar() Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

## Código fuente completo del archivo:

```
package interfaz2;
/**
 *
 * @author alumno
 */
public class SumadorPares {
    private int num1;
    private int num2;
    public SumadorPares(int num1, int num2) {
        this.num1 = num1;
        this.num2 = num2;
    }
    public SumadorPares() {
        num1 = 0;
        num2 = 0;
    }
    public int getNum1() {
        return num1;
    }
    public void setNum1(int num1) {
        this.num1 = num1;
    }
    public int getNum2() {
        return num2;
    }
    public void setNum2(int num2) {
        this.num2 = num2;
    }
    public int sumar() {
        return num1 + num2;
    }
}
```

## VentanaSumador.java — clase VentanaSumador

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JFrame, JLabel, JTextField Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, convierte texto a entero, actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

### Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz2;
/**
 *
 * @author alumno
 */
public class VentanaSumador extends javax.swing.JFrame {
    /**
     * Creates new form VentanaSumador
     */
    public VentanaSumador() {
        initComponents();
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        try {
            int num1 = Integer.parseInt(jTextField1.getText().trim());
            int num2 = Integer.parseInt(jTextField2.getText().trim());
            if (num1 < 0 || num2 < 0) {
                Resultado.setText("Error: Introduce un numero positivo");
            } else {
                SumadorPares sumarPares = new SumadorPares(num1, num2);
                int resultado = sumarPares.sumar();
                Resultado.setText("Resultado: " + resultado);
            }
        } catch (NumberFormatException e) {
            Resultado.setText("Error: Debe de ser un numero entero");
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JLabel Num1;
    private javax.swing.JLabel Num2;
    private javax.swing.JLabel Resultado;
    private javax.swing.JButton jButton1;
    private javax.swing.JTextField jTextField1;
    private javax.swing.JTextField jTextField2;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICA1.2 · APARTADO03

Personaje y ventana Warcraft: objetos con estado y comportamiento.

### Personaje.java — clase Personaje

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Atributos detectados: private String nombre; private int fuerza; private int defensa; private int vida Métodos detectados: getNombre(); setNombre(String nombre); getFuerza(); setFuerza(int fuerza); getDefensa(); setDefensa(int defensa); getVida(); setVida(int vida); toString(); vivo(); ataca(Personaje defensor) Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

## Código fuente completo del archivo:

```
package interfaz3;
import java.util.Random;
/**
 *
 * @author DAVID
 */
public class Personaje {
    private String nombre;
    private int fuerza;
    private int defensa;
    private int vida;
    public Personaje(String nombre, int fuerza, int defensa, int vida) {
        this.nombre = nombre;
        this.fuerza = fuerza;
        this.defensa = defensa;
        this.vida = vida;
    }
    public Personaje() {
        nombre = "";
        fuerza = 0;
        defensa = 0;
        vida = 0;
    }
    //Creo un constructor para poner la Fuerza-Defensa aleatorio y Vida sea 10
    public Personaje(String nombre) {
        this.nombre = nombre;
        Random rand = new Random();
        fuerza = rand.nextInt(8) + 3;
        defensa = rand.nextInt(6);
        vida = 10;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public int getFuerza() {
        return fuerza;
    }
    public void setFuerza(int fuerza) {
        this.fuerza = fuerza;
    }
    public int getDefensa() {
        return defensa;
    }
    public void setDefensa(int defensa) {
        this.defensa = defensa;
    }
    public int getVida() {
        return vida;
    }
    public void setVida(int vida) {
        this.vida = vida;
    }
    @Override
    public String toString() {
        return nombre + " [Fuerza: " + fuerza + ", Defensa: " + defensa + ", Vida: " + vida + "]";
    }
    //Metodo que dice que true si es mas de 0 y false si es 0 o menos
    public boolean vivo() {
        return vida > 0;
    }
    //Metodo que con un Random dice si le da al jugador o falla y si el daño es mejor que 1 se pone a 1
    public String ataca (Personaje defensor) {
        Random rand = new Random();
        int probabilidad = rand.nextInt(100);
        if (probabilidad < 70) {
            int danio = fuerza - defensor.getDefensa();
            if (danio < 1) {
                danio = 1;
            }
            defensor.setVida(defensor.getVida() - danio);
            return nombre + " ataca a " + defensor.getNombre() + " y le quita " + danio + " puntos de
              vida";
        } else {
            return nombre + " ataca pero falla el golpe";
        }
    }
}
```

## VentanaWarcraft.java — clase VentanaWarcraft

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JFrame, JLabel, JScrollPane, JTextArea, JTextField Atributos detectados: private Personaje p1; private Personaje p2 Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actualizarStats(); jButton1ActionPerformed(java.awt.event.ActionEvent evt); jButton2ActionPerformed(java.awt.event.ActionEvent evt); jButton3ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed, jButton2ActionPerformed, jButton3ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JTextArea: área de texto, normalmente dentro de JScrollPane; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz3;
/**
 *
 * @author DAVID
 */
public class VentanaWarcraft extends javax.swing.JFrame {
    /**
     * Creates new form VentanaWarcraft
     */
    public VentanaWarcraft() {
        initComponents();
        //Me lo creo aqui para que no salga hasta que no pongan el nombre de los jugadores
        jButton2.setVisible(false);
        jButton3.setVisible(false);
        //También lo pongo para que no se pueda editar
        jTextArea1.setEditable(false);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    //Me creo dos Objetos de Persona en modo privado para que no se pueda cambiar
    private Personaje p1;
    private Personaje p2;
    //Creo un metodo para que cada vez que ataca o empieza se guarde de esta forma
    private void actualizarStats() {
        jLabel1.setText("Fuerza: " + p1.getFuerza());
        jLabel2.setText("Defensa: " + p1.getDefensa());
        jLabel3.setText("Vida: " + p1.getVida());
        jLabel6.setText("Fuerza: " + p2.getFuerza());
        jLabel5.setText("Defensa: " + p2.getDefensa());
        jLabel4.setText("Vida: " + p2.getVida());
    }
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        String jugador1 = jTextField1.getText().trim();
        String jugador2 = jTextField2.getText().trim();
        if (jugador1.isEmpty() || jugador2.isEmpty()) {
            jTextArea1.setText("Introduce un nombre para cada jugador \n");
            //Para que no se vea los botones
            jButton2.setVisible(false);
            jButton3.setVisible(false);
        } else {
            p1 = new Personaje(jugador1);
            p2 = new Personaje(jugador2);
            actualizarStats();
            jTextArea1.setText("--COMIENZA EL COMBATE-- \n");
            jTextArea1.append(jugador1 + " vs " + jugador2 + "\n\n");
            //Para que se vea los botones
            jButton2.setVisible(true);
            jButton3.setVisible(true);
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton2ActionPerformed
        if (p1 == null || p2 == null) {
        } else {
            String resultado = p1.ataca(p2);
            jTextArea1.append(resultado + "\n");
            actualizarStats();
            if (!p2.vivo()) {
                jTextArea1.append("\n\n --" + p1.getNombre() + " GANA EL COMBATE--");
                //Para que no se vea los botones
                jButton2.setVisible(false);
                jButton3.setVisible(false);
            }
        }
    }//GEN-LAST:event_jButton2ActionPerformed
    private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton3ActionPerformed
        if (p1 == null || p2 == null) {
        } else {
            String resultado = p2.ataca(p1);
            jTextArea1.append(resultado + "\n");
```

```
            actualizarStats();
            if (!p1.vivo()) {
                jTextArea1.append("\n\n --" + p2.getNombre() + " GANA EL COMBATE--");
                //Para que no se vea los botones
                jButton2.setVisible(false);

jButton3.setVisible(false);
            }
        }
    }//GEN-LAST:event_jButton3ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JButton jButton2;
    private javax.swing.JButton jButton3;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JLabel jLabel4;
    private javax.swing.JLabel jLabel5;
    private javax.swing.JLabel jLabel6;
    private javax.swing.JLabel jLabel7;
    private javax.swing.JLabel jLabel8;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTextArea jTextArea1;
    private javax.swing.JTextField jTextField1;
    private javax.swing.JTextField jTextField2;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICA1.3 · APARTADO01

Catálogo de libros: reutilizar una clase de datos y mostrar varios registros.

### Libro.java — clase Libro

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Atributos detectados: private int numPaginas; private String titulo; private boolean almacenado Métodos detectados: getNumPaginas(); setNumPaginas(int numPaginas); getTitulo(); setTitulo(String titulo); isAlmacenado(); setAlmacenado(boolean almacenado); toString() Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

## Código fuente completo del archivo:

```
package interfaz_E1;
/**
 *
 * @author alumno
 */
public class Libro {
    private int numPaginas;
    private String titulo;
    private boolean almacenado;
    public Libro(int numPaginas, String titulo, boolean almacenado) {
        this.numPaginas = numPaginas;
        this.titulo = titulo;
        this.almacenado = almacenado;
    }
    public Libro() {
        numPaginas = 0;
        titulo = "";
        almacenado = false;
    }
    public int getNumPaginas() {
        return numPaginas;
    }
    public void setNumPaginas(int numPaginas) {
        this.numPaginas = numPaginas;
    }
    public String getTitulo() {
        return titulo;
    }
    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }
    public boolean isAlmacenado() {
        return almacenado;
    }
    public void setAlmacenado(boolean almacenado) {
        this.almacenado = almacenado;
    }
    @Override
    public String toString() {
        //Se pone el atributo y detras el simbolo ? y luego separado por dos puntos sale cuando se
        ejecuta el checkBox
        return titulo + " de " + numPaginas + " paginas, " + (almacenado ? "esta" : "no esta") + " en la
        biblioteca";
    }
}
```

## VentanaCatálogo.java — clase VentanaCatálogo

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JCheckBox, JFrame, JLabel, JScrollPane, JTextArea, JTextField Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, convierte texto a entero, consulta una casilla u opción, actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JCheckBox: casilla de verificación; JLabel: etiquetas para títulos e instrucciones; JTextArea: área de texto, normalmente dentro de JScrollPane; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

### Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz_E1;
import java.util.ArrayList;
/**
 *
 * @author alumno
 */
public class VentanaCatálogo extends javax.swing.JFrame {
    ArrayList<Libro> catalogo = new ArrayList<>();
    /**
     * Creates new form VentanaCatálogo
     */
    public VentanaCatálogo() {
        initComponents();
        jTextArea1.setEditable(false);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        try {
            String titulo = jTextField2.getText().trim();
            int paginas = Integer.parseInt(jTextField1.getText().trim());
            boolean estaAlmacenado = jCheckBox1.isSelected();
            if (titulo.isEmpty()) {
                jTextArea1.setText("Escribe un título válido");
            } else if (paginas <= 0) {
                jTextArea1.setText("El número de páginas debe ser positivo");
            } else {
                Libro libro = new Libro(paginas, titulo, estaAlmacenado);
                catalogo.add(libro);
                jTextArea1.setText("--CATÁLOGO ACTUAL--\n");
                for (Libro li : catalogo) {
                    jTextArea1.append(" - " + li.toString() + "\n");
                }
                jTextArea1.append("\nTotal de libros: " + catalogo.size());
            }
        } catch (NumberFormatException e) {
            jTextArea1.setText("El número de páginas debe ser un número entero");
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JCheckBox jCheckBox1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTextArea jTextArea1;
    private javax.swing.JTextField jTextField1;
    private javax.swing.JTextField jTextField2;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICA1.3 · APARTADO03

Combate con personajes especializados: herencia, clases abstractas y polimorfismo.

### Arquero.java — clase Arquero

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Métodos detectados: ataca(Personaje defensor) Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

### Código fuente completo del archivo:

```
package logica;
import java.util.Random;
public class Arquero extends Personaje {
    public Arquero(String nombre) {
        super(nombre);
        setVida(10);
        setVidaMax(10);
    }
    @Override
    public String ataca(Personaje defensor) {
        Random rand = new Random();
        if (rand.nextInt(100) < 90) {
            int danio = calcularDanio(defensor);
            defensor.setVida(defensor.getVida() - danio);
            return getNombre() + " (Arquero) dispara una flecha certera a "
                + defensor.getNombre() + " y le quita " + danio + " de vida";
        }
        return getNombre() + " (Arquero) falla el disparo";
    }
}
```

## Guerrero.java — clase Guerrero

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Métodos detectados: ataca(Personaje defensor) Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

### Código fuente completo del archivo:

```
package logica;
import java.util.Random;
public class Guerrero extends Personaje {
    public Guerrero(String nombre) {
        super(nombre);
        setVida(15);
        setVidaMax(15);
        setDefensa(new Random().nextInt(3));
    }
    @Override
    public String ataca(Personaje defensor) {
        Random rand = new Random();
        if (rand.nextInt(100) < 75) {
            int danio = calcularDanio(defensor) + 1;
            defensor.setVida(defensor.getVida() - danio);
            return getNombre() + " (Guerrero) golpea con su espada a "
                + defensor.getNombre() + " y le quita " + danio + " de vida";
        }
        return getNombre() + " (Guerrero) falla su espadazo";
    }
}
```

## Jefe.java — clase Jefe

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Métodos detectados: ataca(Personaje defensor) Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

## Código fuente completo del archivo:

```
package logica;
import java.util.Random;
public class Jefe extends Personaje {
    public Jefe() {
        super("JEFE");
        setVida(20);
        setVidaMax(20);
        setFuerza(10);
        setDefensa(4);
    }
    @Override
    public String ataca(Personaje defensor) {
        Random rand = new Random();
        if (rand.nextInt(100) < 50) {
            int danio = calcularDanio(defensor) + 2;
            defensor.setVida(defensor.getVida() - danio);
            return getNombre() + " aplasta a " + defensor.getNombre()
                + " y le quita " + danio + " de vida";
        }
        return "- " + getNombre() + " falla su ataque devastador";
    }
}
```

## Mago.java — clase Mago

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Métodos detectados: ataca(Personaje defensor) Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

## Código fuente completo del archivo:

```
package logica;
import java.util.Random;
public class Mago extends Personaje {
    public Mago(String nombre) {
        super(nombre);
        setVida(8);
        setVidaMax(8);
        setFuerza(new Random().nextInt(8) + 5);
    }
    @Override
    public String ataca(Personaje defensor) {
        Random rand = new Random();
        if (rand.nextInt(100) < 80) {
            int danio = getFuerza();
            if (danio < 1) danio = 1;
            defensor.setVida(defensor.getVida() - danio);
            return getNombre() + " (Mago) lanza un hechizo a "
                + defensor.getNombre() + " y le quita " + danio + " de vida";
        }
        return getNombre() + " (Mago) falla su hechizo";
    }
}
```

## Personaje.java — clase Personaje

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Atributos detectados: private String nombre; private int fuerza; private int defensa; private int vida; private int vidaMax Métodos detectados: getNombre(); setNombre(String nombre); getFuerza(); setFuerza(int fuerza); getDefensa(); setDefensa(int defensa); getVida(); setVida(int vida); getVidaMax(); setVidaMax(int vidaMax); toString(); vivo(); ataca(Personaje defensor); calcularDanio(Personaje defensor) Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

## Código fuente completo del archivo:

```
package logica;
import java.util.Random;
/**
 *
 * @author DAVID
 */
public abstract class Personaje {
    private String nombre;
    private int fuerza;
    private int defensa;
    private int vida;
    private int vidaMax;
    public Personaje(String nombre, int fuerza, int defensa, int vida) {
        this.nombre = nombre;
        this.fuerza = fuerza;
        this.defensa = defensa;
        this.vida = vida;
    }
    public Personaje() {
        nombre = "";
        fuerza = 0;
        defensa = 0;
        vida = 0;
    }
    //Creo un constructor para poner la Fuerza-Defensa aleatorio y Vida sea 10
    public Personaje(String nombre) {
        this.nombre = nombre;
        Random rand = new Random();
        fuerza = rand.nextInt(8) + 3;
        defensa = rand.nextInt(6);
        vida = 10;
        vidaMax = 10;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public int getFuerza() {
        return fuerza;
    }
    public void setFuerza(int fuerza) {
        this.fuerza = fuerza;
    }
    public int getDefensa() {
        return defensa;
    }
    public void setDefensa(int defensa) {
        this.defensa = defensa;
    }
    public int getVida() {
        return vida;
    }
    public void setVida(int vida) {
        this.vida = vida;
    }
    public int getVidaMax() {
        return vidaMax;
    }
    public void setVidaMax(int vidaMax) {
        this.vidaMax = vidaMax;
    }
    @Override
    public String toString() {
        return nombre + " [Fuerza: " + fuerza + ", Defensa: " + defensa + ", Vida: " + vida + "]";
    }
    //Metodo que dice que true si es mas de 0 y false si es 0 o menos
    public boolean vivo() {
        return vida > 0;
    }
    public abstract String ataca (Personaje defensor);
    public int calcularDanio(Personaje defensor) {
        int danio = fuerza - defensor.getDefensa();
        return Math.max(danio, 1);
    }
}
```

## VentanaCombate.java — clase VentanaCombate

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JFrame, JLabel, JScrollPane, JTextArea Atributos detectados: private Personaje heroe; private Jefe jefe Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); jButton2ActionPerformed(java.awt.event.ActionEvent evt); actualizarStats(); log(String mensaje); main(String args[]); run() Eventos detectados: jButton1ActionPerformed, jButton2ActionPerformed Cómo funciona en conjunto: El código actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JTextArea: área de texto, normalmente dentro de JScrollPane. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz_E3;
import logica.Jefe;
import logica.Personaje;
/**
 *
 * @author alumno
 */
public class VentanaCombate extends javax.swing.JFrame {
    private Personaje heroe;
    private Jefe jefe;
    /**
     * Creates new form VentanaCombate
     */
    public VentanaCombate(Personaje heroe) {
        this.heroe = heroe;
        jefe = new Jefe();
        initComponents();
        jTextArea1.setEditable(false);
        jButton2.setVisible(false);
        actualizarStats();
        log("¡" + heroe.getNombre() + " el " + heroe.getClass().getSimpleName() + " se enfrenta al " +
        jefe.getNombre() + "!");
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        log(heroe.ataca(jefe));
        actualizarStats();
        if (!jefe.vivo()) {
            log("\n --" + heroe.getNombre() + " GANA LA BATALLA--");
            jButton1.setEnabled(false);
            jButton2.setVisible(true);
        } else {
            log(jefe.ataca(heroe));
            actualizarStats();
            if (!heroe.vivo()) {
                log("\n --" + jefe.getNombre() + " GANA LA BATALLA--");
                jButton1.setEnabled(false);
                jButton2.setVisible(true);
            }
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton2ActionPerformed
        new VentanaSeleccion().setVisible(true);
        this.dispose();
    }//GEN-LAST:event_jButton2ActionPerformed
    private void actualizarStats() {
        jLabel3.setText("Fuerza: " + heroe.getFuerza());
        jLabel4.setText("Defensa: " + heroe.getDefensa());
        jLabel5.setText("Vida: " + heroe.getVida());
        jLabel6.setText("Fuerza: " + jefe.getFuerza());
        jLabel7.setText("Defensa: " + jefe.getDefensa());
        jLabel8.setText("Vida: " + jefe.getVida());
    }
    private void log(String mensaje) {
        jTextArea1.append(mensaje + "\n");
        jTextArea1.setCaretPosition(jTextArea1.getDocument().getLength());
    }
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JButton jButton2;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JLabel jLabel4;
    private javax.swing.JLabel jLabel5;
```

```
    private javax.swing.JLabel jLabel6;
    private javax.swing.JLabel jLabel7;
    private javax.swing.JLabel jLabel8;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTextArea jTextArea1;
    // End of variables declaration//GEN-END:variables
}
```

## VentanaSeleccion.java — clase VentanaSeleccion

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JComboBox, JFrame, JLabel, JTextField Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JComboBox: lista desplegable con las opciones del ejercicio; JLabel: etiquetas para títulos e instrucciones; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

### Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz_E3;
import logica.Arquero;
import logica.Guerrero;
import logica.Mago;
import logica.Personaje;
/**
 *
 * @author alumno
 */
public class VentanaSeleccion extends javax.swing.JFrame {
    /**
     * Creates new form VentanaSeleccion
     */
    public VentanaSeleccion() {
        initComponents();
        jLabel3.setText("");
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        String nombre = jTextField1.getText().trim();
        String clase = (String) jComboBox1.getSelectedItem();
        if (nombre.isEmpty()) {
            jLabel3.setText("Error: Escribe un nombre para continuar");
        } else {
            jLabel3.setText("");
            Personaje miPersonaje;
            if (clase.equals("Guerrero")) {
                miPersonaje = new Guerrero(nombre);
            } else if (clase.equals("Mago")) {
                miPersonaje = new Mago(nombre);
            } else {
                miPersonaje = new Arquero(nombre);
            }
            new VentanaCombate(miPersonaje).setVisible(true);
            this.dispose();
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JComboBox<String> jComboBox1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JTextField jTextField1;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICA1.4

Juegos y ventana principal: tablero, eventos, turnos, puntuación y navegación.

### Jugador.java — clase Jugador

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Atributos detectados: private String nombre; private int puntuacion Métodos detectados: getNombre(); setNombre(String nombre); getPuntuacion(); setPuntuacion(int puntuacion); toString() Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

## Código fuente completo del archivo:

```
package logica;
/**
 *
 * @author DAVID
 */
public class Jugador {
    private String nombre;
    private int puntuacion;
    public Jugador(String nombre) {
        this.nombre = nombre;
        puntuacion = 0;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public int getPuntuacion() {
        return puntuacion;
    }
    public void setPuntuacion(int puntuacion) {
        this.puntuacion = puntuacion;
    }
    @Override
    public String toString() {
        return "Jugador - Nombre: " + nombre + " | Puntuacion: " + puntuacion;
    }
}
```

## VentanaBuscaminas.java — clase VentanaBuscaminas

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JFrame, JLabel, JPanel Atributos detectados: private int minasDescubiertas = 0 Métodos detectados: initComponents(); mouseClicked(java.awt.event.MouseEvent evt); mouseEntered(java.awt.event.MouseEvent evt); mouseExited(java.awt.event.MouseEvent evt); mouseClicked(java.awt.event.MouseEvent evt); mouseEntered(java.awt.event.MouseEvent evt); mouseExited(java.awt.event.MouseEvent evt); mouseClicked(java.awt.event.MouseEvent evt); mouseEntered(java.awt.event.MouseEvent evt); mouseExited(java.awt.event.MouseEvent evt); mouseClicked(java.awt.event.MouseEvent evt); mouseEntered(java.awt.event.MouseEvent evt); mouseExited(java.awt.event.MouseEvent evt); mouseClicked(java.awt.event.MouseEvent evt); mouseEntered(java.awt.event.MouseEvent evt); mouseExited(java.awt.event.MouseEvent evt) Eventos detectados: jLabel2MouseClicked Cómo funciona en conjunto: El código lee texto introducido, actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JLabel: etiquetas para títulos e instrucciones; JPanel: paneles para agrupar y ordenar controles. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
import java.awt.Color;
import javax.swing.JLabel;
import javax.swing.SwingConstants;
/**
 *
 * @author DAVID
 */
public class VentanaBuscaminas extends javax.swing.JFrame {
    private int minasDescubiertas = 0;
    /**
     * Creates new form VentanaBuscaminas
     */
    public VentanaBuscaminas() {
        initComponents();
        this.setSize(500, 500);
        this.setLocationRelativeTo(null);
        JLabel[] etiquetas = {jLabel2, jLabel3, jLabel4, jLabel5, jLabel6, jLabel7, jLabel8, jLabel9,
          jLabel10};
        for (JLabel lbl : etiquetas) {
            lbl.setOpaque(true);
            lbl.setBackground(Color.BLUE);
            lbl.setHorizontalAlignment(SwingConstants.CENTER);
            lbl.setName(lbl.getText());
        }
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jLabel2MouseClicked(java.awt.event.MouseEvent evt) {//GEN-
      FIRST:event_jLabel2MouseClicked
        JLabel label = (JLabel) evt.getSource();
        if (!label.getText().equals("¡BOOM!")) {
            label.setBackground(Color.RED);
            label.setText("¡BOOM!");
            minasDescubiertas++;
            jLabel1.setText("Minas encontradas: " + minasDescubiertas);
        }
    }//GEN-LAST:event_jLabel2MouseClicked
    private void jLabel2MouseEntered(java.awt.event.MouseEvent evt) {//GEN-
      FIRST:event_jLabel2MouseEntered
        JLabel label = (JLabel) evt.getSource();
        if (!label.getText().equals("¡BOOM!")) {
            label.setBackground(Color.YELLOW);
            label.setText("?");
        }
    }//GEN-LAST:event_jLabel2MouseEntered
    private void jLabel2MouseExited(java.awt.event.MouseEvent evt) {//GEN-FIRST:event_jLabel2MouseExited
        JLabel label = (JLabel) evt.getSource();
        if (!label.getText().equals("¡BOOM!")) {
            label.setBackground(Color.BLUE);
            label.setText(label.getName());
        }
    }//GEN-LAST:event_jLabel2MouseExited
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel10;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JLabel jLabel4;
    private javax.swing.JLabel jLabel5;
    private javax.swing.JLabel jLabel6;
    private javax.swing.JLabel jLabel7;
    private javax.swing.JLabel jLabel8;
    private javax.swing.JLabel jLabel9;
    private javax.swing.JPanel jPanel1;
    // End of variables declaration//GEN-END:variables
}
```

## VentanaCazatesoros.java — clase VentanaCazatesoros

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JFrame, JLabel, JOptionPane, JPanel Atributos detectados: private boolean turnoUsuario = true; private int puntosUsuario = 0; private int puntosCPU = 0; private int tesorosRestantes = 5; private JButton[] botones Métodos detectados: actionPerformed(ActionEvent evt); windowClosing(WindowEvent e); esconderTesoros(); actualizarMarcador(); jugarCPU(); terminarJuego(); initComponents(); actionPerformed(java.awt.event.ActionEvent evt); botonPulsado(java.awt.event.ActionEvent evt); main(String args[]); run() Cómo funciona en conjunto: El código actualiza texto visible, muestra un mensaje al usuario, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JPanel: paneles para agrupar y ordenar controles. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
import java.awt.Color;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.util.Random;
import javax.swing.JButton;
import javax.swing.JOptionPane;
/**
 *
 * @author DAVID
 */
public class VentanaCazatesoros extends javax.swing.JFrame {
    private boolean turnoUsuario = true;
    private int puntosUsuario = 0;
    private int puntosCPU = 0;
    private int tesorosRestantes = 5;
    private JButton[] botones;
    /**
     * Creates new form VentanaCazatesoros
     */
    public VentanaCazatesoros() {
        initComponents();
        this.setSize(500, 500);
        this.setLocationRelativeTo(null);
        botones = new JButton[] {jButton1, jButton2, jButton3, jButton4, jButton5, jButton6, jButton7,
          jButton8,
          jButton9, jButton10, jButton11, jButton12, jButton13, jButton14, jButton15, jButton16};
        for (JButton button : botones) {
            button.setText("");
            button.setBackground(null);
            button.setOpaque(true);
            button.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent evt) {
                    botonPulsado(evt);
                }
            });
        }
        esconderTesoros();
        actualizarMarcador();
        this.addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                new VentanaPrincipal().setVisible(true);
                dispose();
            }
        });
    }
    private void esconderTesoros() {
        Random rand = new Random();
        int tesorosEscondidos = 0;
        while (tesorosEscondidos < 5) {
            int posicionAleatoria = rand.nextInt(16);
            if (botones[posicionAleatoria].getName() == null) {
                botones[posicionAleatoria].setName("tesoro");
                tesorosEscondidos++;
            }
        }
    }
    private void actualizarMarcador() {
        jLabel2.setText("Puntos Usuario: " + puntosUsuario + " || Puntos CPU: " + puntosCPU);
    }
    private void jugarCPU() {
        Random rand = new Random();
        int pos = rand.nextInt(16);
        while (!botones[pos].isEnabled()) {
            pos = rand.nextInt(16);
        }
        JButton buttonCPU = botones[pos];
        if ("tesoro".equals(buttonCPU.getName())) {
            buttonCPU.setBackground(Color.YELLOW);
            puntosCPU++;
            tesorosRestantes--;
        } else {
            buttonCPU.setBackground(Color.DARK_GRAY);
        }
        buttonCPU.setEnabled(false);
```

```
actualizarMarcador();
        turnoUsuario = true;
        if (tesorosRestantes == 0) {
            terminarJuego();
        }
    }
    private void terminarJuego() {
        String ganador;
        if (puntosUsuario > puntosCPU) {
            ganador = "--HAS GANADOO--";
        } else if (puntosCPU > puntosUsuario) {
            ganador = "--CPU HA GANADO--";
        } else {
            ganador = "--EMPATE--";
        }
        JOptionPane.showMessageDialog(this, "FIN DEL JUEGO\n" + ganador + "\nPuntos Usuario: " +
        puntosUsuario + " | Puntos CPU: " + puntosCPU);
        for (JButton button3 : botones) {
            button3.setEnabled(false);
        }
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void botonPulsado(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_botonPulsado
        JButton button16 = (JButton) evt.getSource();
        if (!turnoUsuario || !button16.isEnabled()) {
        } else {
            if ("tesoro".equals(button16.getName())) {
                button16.setBackground(Color.ORANGE);
                puntosUsuario++;
                tesorosRestantes--;
            } else {
                button16.setBackground(Color.DARK_GRAY);
            }
            button16.setEnabled(false);
            actualizarMarcador();
            if (tesorosRestantes == 0) {
                terminarJuego();
            } else {
                turnoUsuario = false;
                jugarCPU();
            }
        }
    }//GEN-LAST:event_botonPulsado
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JButton jButton10;
    private javax.swing.JButton jButton11;
    private javax.swing.JButton jButton12;
    private javax.swing.JButton jButton13;
    private javax.swing.JButton jButton14;
    private javax.swing.JButton jButton15;
    private javax.swing.JButton jButton16;
    private javax.swing.JButton jButton2;
    private javax.swing.JButton jButton3;
    private javax.swing.JButton jButton4;
    private javax.swing.JButton jButton5;
    private javax.swing.JButton jButton6;
    private javax.swing.JButton jButton7;
    private javax.swing.JButton jButton8;
    private javax.swing.JButton jButton9;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JPanel jPanel2;
    // End of variables declaration//GEN-END:variables
}
```

## VentanaPrincipal.java — clase VentanaPrincipal

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JFrame, JLabel, JMenu, JMenuBar, JMenuItem, JOptionPane Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); jMenuItemRegisActionPerformed(java.awt.event.ActionEvent evt); jMenuItemJugarActionPerformed(java.awt.event.ActionEvent evt); jMenuItemSalirActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jMenuItemJugarActionPerformed, jMenuItemRegisActionPerformed, jMenuItemSalirActionPerformed Cómo funciona en conjunto: El código actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JLabel: etiquetas para títulos e instrucciones; JMenuBar: barra de menú; JMenuItem: opciones dentro de un menú. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
import javax.swing.JOptionPane;
import logica.Jugador;
/**
 *
 * @author DAVID
 */
public class VentanaPrincipal extends javax.swing.JFrame {
    /**
     * Creates new form VentanaPrincipal
     */
    public VentanaPrincipal() {
        initComponents();
        this.setSize(500, 500);
        this.setLocationRelativeTo(null);
        jMenuItemJugar.setEnabled(false);
        //Sirve para centrar
        jLabel1.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jMenuItemRegisActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jMenuItemRegisActionPerformed
        String nombre = JOptionPane.showInputDialog(this, "Introduce tu nombre: ");
        if (nombre != null && !nombre.trim().isEmpty()) {
            Jugador miJugador = new Jugador(nombre.trim());
            jLabel1.setText("Vamos a jugar, " + miJugador.getNombre());
            jMenuItemJugar.setEnabled(true);
        }
    }//GEN-LAST:event_jMenuItemRegisActionPerformed
    private void jMenuItemJugarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jMenuItemJugarActionPerformed
        new VentanaCazatesoros().setVisible(true);
        this.setVisible(false);
    }//GEN-LAST:event_jMenuItemJugarActionPerformed
    private void jMenuItemSalirActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jMenuItemSalirActionPerformed
        System.exit(0);
    }//GEN-LAST:event_jMenuItemSalirActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JLabel jLabel1;
    private javax.swing.JMenu jMenu1;
    private javax.swing.JMenuBar jMenuBar1;
    private javax.swing.JMenuItem jMenuItemJugar;
    private javax.swing.JMenuItem jMenuItemRegis;
    private javax.swing.JMenuItem jMenuItemSalir;
    private javax.swing.JMenu jMenuJuegos;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICA1.5 · APARTADO01

Perfil con ventana secundaria: JDialog, validación y comunicación con JFrame.

## DialogPerfil.java — clase DialogPerfil

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JDialog, JFrame, JLabel, JOptionPane, JPanel, JTextField Atributos detectados: private VentanaPrincipal ventanaPadre Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run(); windowClosing(java.awt.event.WindowEvent e) Eventos detectados: jButton1ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, actualiza texto visible, muestra un mensaje al usuario, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JPanel: paneles para agrupar y ordenar controles; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo). Al abrirlo: desde la ventana principal crea el diálogo con new NombreDialogo(this, true), céntralo con setLocationRelativeTo(this) y muéstralo con setVisible(true). Para cerrarlo desde Guardar o Cancelar usa dispose().

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JDialog.java to edit this template
 */
package interfaz_EJ01;
import javax.swing.JOptionPane;
/**
 *
 * @author DAVID
 */
public class DialogPerfil extends javax.swing.JDialog {
    private VentanaPrincipal ventanaPadre;
    /**
     * Creates new form DialogPerfil
     */
    public DialogPerfil(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
        this.setSize(414, 150);
        this.setLocationRelativeTo(null);
        this.ventanaPadre = (VentanaPrincipal) parent;
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        String nombre = jTextField1.getText().trim();
        int nivel = (int) jSpinner1.getValue();
        if (nombre.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Pon tu nombre", "ERROR", JOptionPane.ERROR_MESSAGE);
        } else if (nivel < 0) {
            JOptionPane.showMessageDialog(this, "Tiene que ser un número positivo", "ERROR",
            JOptionPane.ERROR_MESSAGE);
        } else {
            ventanaPadre.actualizarPerfil(nombre, nivel);
            this.dispose();
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JPanel jPanel2;
    private javax.swing.JSpinner jSpinner1;
    private javax.swing.JTextField jTextField1;
    // End of variables declaration//GEN-END:variables
}
```

## VentanaPrincipal.java — clase VentanaPrincipal

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JFrame, JLabel, JPanel Métodos detectados: actualizarPerfil(String nombre, int nivel); initComponents(); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed Cómo funciona en conjunto: El código actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JPanel: paneles para agrupar y ordenar controles.

Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

### Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz_EJ01;
/**
 *
 * @author DAVID
 */
public class VentanaPrincipal extends javax.swing.JFrame {
    /**
     * Creates new form VentanaPrincipal
     */
    public VentanaPrincipal() {
        initComponents();
        this.setSize(414, 150);
        this.setLocationRelativeTo(null);
    }
    public void actualizarPerfil(String nombre, int nivel) {
        jLabel1.setText("Usuario: " + nombre);
        jLabel2.setText("Nivel: " + nivel);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        new DialogPerfil(this, true).setVisible(true);
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JPanel jPanel2;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICA1.5 · APARTADO02

Catálogo de coches: objetos, JTable y modelo personalizado.

### Coche.java — clase Coche

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Atributos detectados: private String marca; private String modelo; private int caballos Métodos detectados: getMarca(); setMarca(String marca); getModelo(); setModelo(String modelo); getCaballos(); setCaballos(int caballos); toString() Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

## Código fuente completo del archivo:

```
package logica_EJ02;
/**
 *
 * @author DAVID
 */
public class Coche {
    private String marca;
    private String modelo;
    private int caballos;
    public Coche(String marca, String modelo, int caballos) {
        this.marca = marca;
        this.modelo = modelo;
        this.caballos = caballos;
    }
    public Coche() {
        marca = "";
        modelo = "";
        caballos = 0;
    }
    public String getMarca() {
        return marca;
    }
    public void setMarca(String marca) {
        this.marca = marca;
    }
    public String getModelo() {
        return modelo;
    }
    public void setModelo(String modelo) {
        this.modelo = modelo;
    }
    public int getCaballos() {
        return caballos;
    }
    public void setCaballos(int caballos) {
        this.caballos = caballos;
    }
    @Override
    public String toString() {
        return "Coche - Marca: " + marca + " | Modelo: " + modelo + " | Caballos: " + caballos;
    }
}
```

## Modelo.java — clase Modelo

Modelo de tabla. Traduce una colección de objetos a filas, columnas y valores que JTable puede mostrar. Componentes o clases Swing detectados: AbstractTableModel Atributos detectados: private List<Coche> listaCoches; private String[] columnas = {"Marca", "Modelo", "Caballos"} Métodos detectados: getListaCoches(); setListaCoches(List<Coche> listaCoches); getColumnas(); setColumnas(String[] columnas); getListenerList(); setListenerList(EventListenerList listenerList); getRowCount(); getColumnCount(); getColumnName(int column); getValueAt(int rowIndex, int columIndex) Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. Para JTable: revisa getRowCount(), getColumnCount(), getColumnName() y getValueAt(). Cuando cambie la colección, notifica a la tabla mediante fireTableDataChanged() o el evento de filas apropiado.

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
 */
package logica_EJ02;
import java.util.List;
import java.util.ArrayList;
import javax.swing.event.EventListenerList;
import javax.swing.table.AbstractTableModel;
/**
 *
 * @author DAVID
 */
public class Modelo extends AbstractTableModel{
    private List<Coche> listaCoches;
    private String[] columnas = {"Marca", "Modelo", "Caballos"};
    public Modelo() {
        listaCoches = new ArrayList<>();
        listaCoches.add(new Coche("Ford", "Focus", 120));
        listaCoches.add(new Coche("Seat", "Ibiza", 110));
        listaCoches.add(new Coche("Ferrari", "F40", 500));
        listaCoches.add(new Coche("BYD", "SEAL U DM-i", 218));
    }
    public List<Coche> getListaCoches() {
        return listaCoches;
    }
    public void setListaCoches(List<Coche> listaCoches) {
        this.listaCoches = listaCoches;
    }
    public String[] getColumnas() {
        return columnas;
    }
    public void setColumnas(String[] columnas) {
        this.columnas = columnas;
    }
    public EventListenerList getListenerList() {
        return listenerList;
    }
    public void setListenerList(EventListenerList listenerList) {
        this.listenerList = listenerList;
    }
    @Override
    public int getRowCount() {
        return listaCoches.size();
    }
    @Override
    public int getColumnCount() {
        return columnas.length;
    }
    @Override
    public String getColumnName(int column) {
        return columnas[column];
    }
    @Override
    public Object getValueAt(int rowIndex, int columIndex) {
        Coche c = listaCoches.get(rowIndex);
        switch (columIndex) {
            case 0: return c.getMarca();
            case 1: return c.getModelo();
            case 2: return c.getCaballos();
            default: return null;
        }
    }
}
```

## VentanaCatalogo.java — clase VentanaCatalogo

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: DefaultTableModel, JFrame, JScrollPane, JTable Métodos detectados: initComponents(); main(String args[]); run() Cómo funciona en conjunto: El código muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JTable: tabla dentro de JScrollPane. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

### Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz_EJ02;
import logica_EJ02.Modelo;
/**
 *
 * @author DAVID
 */
public class VentanaCatalogo extends javax.swing.JFrame {
    /**
     * Creates new form VentanaCatalogo
     */
    public VentanaCatalogo() {
        initComponents();
        this.setSize(500, 500);
        this.setLocationRelativeTo(null);
        Modelo modelo = new Modelo();
        jTable1.setModel(modelo);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTable jTable1;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICA1.5 · APARTADO03

Agenda de contactos: diálogo de alta, validación y tabla.

### Contacto.java — clase Contacto

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Atributos detectados: private String nombre; private String telefono Métodos detectados: getNombre(); setNombre(String nombre); getTelefono(); setTelefono(String telefono) Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

## Código fuente completo del archivo:

```
package logica_EJ03;
/**
 *
 * @author DAVID
 */
public class Contacto {
    private String nombre;
    private String telefono;
    public Contacto(String nombre, String telefono) {
        this.nombre = nombre;
        this.telefono = telefono;
    }
    public Contacto() {
        nombre = "";
        telefono = "";
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public String getTelefono() {
        return telefono;
    }
    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }
}
```

## DialogContacto.java — clase DialogContacto

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JDialog, JFrame, JLabel, JOptionPane, JPanel, JTextField Atributos detectados: private VentanaAgenda ventanaPadre Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run(); windowClosing(java.awt.event.WindowEvent e) Eventos detectados: jButton1ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, actualiza texto visible, muestra un mensaje al usuario, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JPanel: paneles para agrupar y ordenar controles; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo). Al abrirlo: desde la ventana principal crea el diálogo con new NombreDialogo(this, true), céntralo con setLocationRelativeTo(this) y muéstralo con setVisible(true). Para cerrarlo desde Guardar o Cancelar usa dispose().

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JDialog.java to edit this template
 */
package interfaz_EJ03;
import javax.swing.JOptionPane;
import logica_EJ03.Contacto;
/**
 *
 * @author DAVID
 */
public class DialogContacto extends javax.swing.JDialog {
    private VentanaAgenda ventanaPadre;
    /**
     * Creates new form DialogContacto
     */
    public DialogContacto(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
        this.setSize(414, 150);
        this.setLocationRelativeTo(null);
        this.ventanaPadre = (VentanaAgenda) parent;
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        String nombre = jTextField1.getText().trim();
        String telefono = jTextField2.getText().trim();
        if (nombre.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Rellena el nombre", "Error", JOptionPane.ERROR_MESSAGE);
        } else if (telefono.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Rellena el telefono", "Error",
              JOptionPane.ERROR_MESSAGE);
        } else if (telefono.length() != 9) {
            JOptionPane.showMessageDialog(this, "Debe tener 9 números", "Error",
              JOptionPane.ERROR_MESSAGE);
        }
        else {
            Contacto c = new Contacto(nombre, telefono);
            ventanaPadre.recibirContacto(c);
            this.dispose();
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JPanel jPanel2;
    private javax.swing.JTextField jTextField1;
    private javax.swing.JTextField jTextField2;
    // End of variables declaration//GEN-END:variables
}
```

## Modelo.java — clase Modelo

Modelo de tabla. Traduce una colección de objetos a filas, columnas y valores que JTable puede mostrar. Componentes o clases Swing detectados: AbstractTableModel Atributos detectados: private List<Contacto> listaContactos = new ArrayList<>(); private String[] columnas = {"Nombre", "Telefono"} Métodos detectados: addModelo(Contacto c); getRowCount(); getColumnCount(); getColumnName(int col); getValueAt(int row, int col) Cómo funciona en conjunto: El código notifica cambios a JTable. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. Para JTable: revisa getRowCount(), getColumnCount(), getColumnName() y getValueAt(). Cuando cambie la colección, notifica a la tabla mediante fireTableDataChanged() o el evento de filas apropiado.

## Código fuente completo del archivo:

```
package logica_EJ03;
import java.util.ArrayList;
import java.util.List;
import javax.swing.table.AbstractTableModel;
/**
 *
 * @author DAVID
 */
public class Modelo extends AbstractTableModel{
    private List<Contacto> listaContactos = new ArrayList<>();
    private String[] columnas = {"Nombre", "Telefono"};
    public void addModelo(Contacto c) {
        listaContactos.add(c);
        this.fireTableDataChanged();
    }
    @Override
    public int getRowCount() {
        return listaContactos.size();
    }
    @Override
    public int getColumnCount() {
        return columnas.length;
    }
    @Override
    public String getColumnName(int col) {
        return columnas[col];
    }
    @Override
    public Object getValueAt(int row, int col) {
        Contacto c = listaContactos.get(row);
        switch (col) {
            case 0: return c.getNombre();
            case 1: return c.getTelefono();
            default: return null;
        }
    }
}
```

## VentanaAgenda.java — clase VentanaAgenda

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: DefaultTableModel, JButton, JFrame, JPanel, JScrollPane, JTable Atributos detectados: private Modelo modelo Métodos detectados: recibirContacto(Contacto c); initComponents(); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed Cómo funciona en conjunto: El código actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JPanel: paneles para agrupar y ordenar controles; JTable: tabla dentro de JScrollPane. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

### Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz_EJ03;
import logica_EJ03.Contacto;
import logica_EJ03.Modelo;
/**
 *
 * @author DAVID
 */
public class VentanaAgenda extends javax.swing.JFrame {
    private Modelo modelo;
    /**
     * Creates new form VentanaAgenda
     */
    public VentanaAgenda() {
        initComponents();
        this.setSize(500, 500);
        this.setLocationRelativeTo(null);
        modelo = new Modelo();
        jTable1.setModel(modelo);
    }
    public void recibirContacto(Contacto c) {
        modelo.addModelo(c);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        new DialogContacto(this, true).setVisible(true);
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JPanel jPanel2;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTable jTable1;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICA1.5 · APARTADO04

Reclutamiento de astronautas: diálogo, objetos y modelo de tabla.

### Astronauta.java — clase Astronauta

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Atributos detectados: private String nombre; private int misiones Métodos detectados: getNombre(); setNombre(String nombre); getMisiones(); setMisiones(int misiones) Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

## Código fuente completo del archivo:

```
package logica_EJ04;
/**
 *
 * @author DAVID
 */
public class Astronauta {
    private String nombre;
    private int misiones;
    public Astronauta(String nombre, int misiones) {
        this.nombre = nombre;
        this.misiones = misiones;
    }
    public Astronauta() {
        nombre = "";
        misiones = 0;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public int getMisiones() {
        return misiones;
    }
    public void setMisiones(int misiones) {
        this.misiones = misiones;
    }
}
```

## DialogReclutamiento.java — clase DialogReclutamiento

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JDialog, JFrame, JLabel, JOptionPane, JPanel, JTextField Atributos detectados: private VentanaNave ventanaPadre Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run(); windowClosing(java.awt.event.WindowEvent e) Eventos detectados: jButton1ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, actualiza texto visible, muestra un mensaje al usuario, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JPanel: paneles para agrupar y ordenar controles; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo). Al abrirlo: desde la ventana principal crea el diálogo con new NombreDialogo(this, true), céntralo con setLocationRelativeTo(this) y muéstralo con setVisible(true). Para cerrarlo desde Guardar o Cancelar usa dispose().

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JDialog.java to edit this template
 */
package interfaz_EJ04;
import javax.swing.JOptionPane;
import logica_EJ04.Astronauta;
/**
 *
 * @author DAVID
 */
public class DialogReclutamiento extends javax.swing.JDialog {
    private VentanaNave ventanaPadre;
    /**
     * Creates new form DialogReclutamiento
     */
    public DialogReclutamiento(java.awt.Frame parent, boolean modal) {
        super(parent, modal);
        initComponents();
        this.setSize(414, 150);
        this.setLocationRelativeTo(null);
        this.ventanaPadre = (VentanaNave) parent;
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        String nombre = jTextField1.getText().trim();
        int misiones = (int) jSpinner1.getValue();
        if (nombre.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Rellena el nombre", "Error", JOptionPane.ERROR_MESSAGE);
        } else if (misiones < 0) {
            JOptionPane.showMessageDialog(this, "Las misiones deben ser positivos", "Error",
            JOptionPane.ERROR_MESSAGE);
        } else {
            Astronauta a = new Astronauta(nombre, misiones);
            ventanaPadre.recibirAstronauta(a);
            this.dispose();
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JPanel jPanel2;
    private javax.swing.JPanel jPanel4;
    private javax.swing.JSpinner jSpinner1;
    private javax.swing.JTextField jTextField1;
    // End of variables declaration//GEN-END:variables
}
```

## Modelo.java — clase Modelo

Modelo de tabla. Traduce una colección de objetos a filas, columnas y valores que JTable puede mostrar. Componentes o clases Swing detectados: AbstractTableModel Atributos detectados: private List<Astronauta> listaAstronautas = new ArrayList<>(); private String[] columnas = {"Nombre", "Misiones", "Categoría"} Métodos detectados: addModelo(Astronauta a); getRowCount(); getColumnCount(); getColumnName(int col); getValueAt(int row, int col) Cómo funciona en conjunto: El código notifica cambios a JTable. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. Para JTable: revisa getRowCount(), getColumnCount(), getColumnName() y getValueAt(). Cuando cambie la colección, notifica a la tabla mediante fireTableDataChanged() o el evento de filas apropiado.

## Código fuente completo del archivo:

```
package logica_EJ04;
import java.util.ArrayList;
import java.util.List;
import javax.swing.table.AbstractTableModel;
/**
 *
 * @author DAVID
 */
public class Modelo extends AbstractTableModel{
    private List<Astronauta> listaAstronautas = new ArrayList<>();
    private String[] columnas = {"Nombre", "Misiones", "Categoría"};
    public void addModelo(Astronauta a) {
        listaAstronautas.add(a);
        this.fireTableDataChanged();
    }
    @Override
    public int getRowCount() {
        return listaAstronautas.size();
    }
    @Override
    public int getColumnCount() {
        return columnas.length;
    }
    @Override
    public String getColumnName(int col) {
        return columnas[col];
    }
    @Override
    public Object getValueAt(int row, int col) {
        Astronauta a = listaAstronautas.get(row);
        switch (col) {
            case 0: return a.getNombre();
            case 1: return a.getMisiones();
            case 2: return (a.getMisiones() >= 3) ? "Veterano" : "Aprendiz";
            default: return null;
        }
    }
}
```

## VentanaNave.java — clase VentanaNave

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: DefaultTableModel, JButton, JFrame, JPanel, JScrollPane, JTable Atributos detectados: private Modelo modelo Métodos detectados: recibirAstronauta(Astronauta a); initComponents(); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed Cómo funciona en conjunto: El código actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JPanel: paneles para agrupar y ordenar controles; JTable: tabla dentro de JScrollPane. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

### Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz_EJ04;
import logica_EJ04.Astronauta;
import logica_EJ04.Modelo;
/**
 *
 * @author DAVID
 */
public class VentanaNave extends javax.swing.JFrame {
    private Modelo modelo;
    /**
     * Creates new form VentanaNave
     */
    public VentanaNave() {
        initComponents();
        this.setSize(500, 500);
        this.setLocationRelativeTo(null);
        modelo = new Modelo();
        jTable1.setModel(modelo);
    }
    public void recibirAstronauta(Astronauta a) {
        modelo.addModelo(a);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        new DialogReclutamiento(this, true).setVisible(true);
    }//GEN-LAST:event_jButton1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTable jTable1;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICAEXTRA · APARTADO01

Registro de productos: formularios, menús y selección de categorías.

### Producto.java — clase Producto

Clase de datos o lógica. Agrupa atributos y operaciones que utiliza la interfaz. Componentes o clases Swing detectados: No usa componentes Swing directamente; es una clase de lógica/datos. Atributos detectados: private String nombre; private int cantidad; private String categoria Métodos detectados: getNombre(); setNombre(String nombre); getCantidad(); setCantidad(int cantidad); getCategoria(); setCategoria(String categoria); toString() Cómo funciona en conjunto: El código define o consulta el estado del objeto, proporciona datos o comportamiento a otras clases. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana.

### Código fuente completo del archivo:

```
package logica;
/**
 *
 * @author alumno
 */
public class Producto {
    private String nombre;
    private int cantidad;
    private String categoria;
    public Producto(String nombre, int cantidad, String categoria) {
        this.nombre = nombre;
        this.cantidad = cantidad;
        this.categoria = categoria;
    }
    public Producto() {
        nombre = "";
        cantidad = 0;
        categoria = "";
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public int getCantidad() {
        return cantidad;
    }
    public void setCantidad(int cantidad) {
        this.cantidad = cantidad;
    }
    public String getCategoria() {
        return categoria;
    }
    public void setCategoria(String categoria) {
        this.categoria = categoria;
    }
    @Override
    public String toString() {
        return "Producto - Nombre: " + nombre + " | Cantidad: " + cantidad + " | Categoria: " +
          categoria;
    }
}
```

## VentanaPrincipal.java — clase VentanaPrincipal

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JFrame, JMenu, JMenuBar, JMenuItem Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); jMenuItem2ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jMenuItem2ActionPerformed Cómo funciona en conjunto: El código actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JMenuBar: barra de menú; JMenuItem: opciones dentro de un menú. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
/**
 *
 * @author alumno
 */
public class VentanaPrincipal extends javax.swing.JFrame {
    /**
     * Creates new form VentanaPrincipal
     */
    public VentanaPrincipal() {
        initComponents();
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jMenuItem2ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jMenuItem2ActionPerformed
        new VentanaRegistro().setVisible(true);
    }//GEN-LAST:event_jMenuItem2ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JMenu jMenu1;
    private javax.swing.JMenu jMenu2;
    private javax.swing.JMenuBar jMenuBar1;
    private javax.swing.JMenuItem jMenuItem1;
    private javax.swing.JMenuItem jMenuItem2;
    // End of variables declaration//GEN-END:variables
}
```

## VentanaRegistro.java — clase VentanaRegistro

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: ButtonGroup, JButton, JFrame, JLabel, JOptionPane, JRadioButton, JTextField Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); jRadioButton1ActionPerformed(java.awt.event.ActionEvent evt); jRadioButton2ActionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); jRadioButton3ActionPerformed(java.awt.event.ActionEvent evt); jButton2ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed, jButton2ActionPerformed, jRadioButton1ActionPerformed, jRadioButton2ActionPerformed, jRadioButton3ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, consulta una casilla u opción, actualiza texto visible, muestra un mensaje al usuario, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JRadioButton: botones de opción agrupados con ButtonGroup; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
import javax.swing.JOptionPane;
import logica.Producto;
/**
 *
 * @author alumno
 */
public class VentanaRegistro extends javax.swing.JFrame {
    /**
     * Creates new form VentanaRegistro
     */
    public VentanaRegistro() {
        initComponents();
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jRadioButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jRadioButton1ActionPerformed
        // TODO add your handling code here:
    }//GEN-LAST:event_jRadioButton1ActionPerformed
    private void jRadioButton2ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jRadioButton2ActionPerformed
        // TODO add your handling code here:
    }//GEN-LAST:event_jRadioButton2ActionPerformed
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        String nombre = jTextField1.getText().trim();
        if (nombre.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Nombre de producto obligatorio", "Error",
            JOptionPane.ERROR_MESSAGE);
        } else {
            int cantidad = (int) jSpinner1.getValue();
            String categoria = "";
            if (jRadioButton1.isSelected()) {
                categoria = categoria + "Alimentación ";
            }
            if (jRadioButton2.isSelected()) {
                categoria = categoria + "Limpieza ";
            }
            if (jRadioButton3.isSelected()) {
                categoria = categoria + "Electrónica ";
            }
            if (jRadioButton4.isSelected()) {
                categoria = categoria + "Sin categoría ";
            }
            Producto p = new Producto(nombre, cantidad, categoria);
            System.out.println("Producto guardado - Nombre: " + p.getNombre() + " | Cantidad: " +
            p.getCantidad() + " | Categoria: " + p.getCategoria());
            this.dispose();
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    private void jRadioButton3ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jRadioButton3ActionPerformed
        // TODO add your handling code here:
    }//GEN-LAST:event_jRadioButton3ActionPerformed
    private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton2ActionPerformed
        this.dispose();
    }//GEN-LAST:event_jButton2ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.ButtonGroup buttonGroup;
    private javax.swing.JButton jButton1;
    private javax.swing.JButton jButton2;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JRadioButton jRadioButton1;
    private javax.swing.JRadioButton jRadioButton2;
```

```
    private javax.swing.JRadioButton jRadioButton3;
    private javax.swing.JRadioButton jRadioButton4;
    private javax.swing.JSpinner jSpinner1;
    private javax.swing.JTextField jTextField1;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICAEXTRA · APARTADO02

Tablero por turnos: botones, estado y eventos.

### VentanaTablero.java — clase VentanaTablero

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JFrame, JLabel, JPanel Atributos detectados: private boolean turno = true Métodos detectados: pulsarCasilla(javax.swing.JButton boton); initComponents(); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); jButton2ActionPerformed(java.awt.event.ActionEvent evt); jButton4ActionPerformed(java.awt.event.ActionEvent evt); jButton3ActionPerformed(java.awt.event.ActionEvent evt); jButton5ActionPerformed(java.awt.event.ActionEvent evt) Eventos detectados: jButton1ActionPerformed, jButton2ActionPerformed, jButton3ActionPerformed, jButton4ActionPerformed, jButton5ActionPerformed, jButton6ActionPerformed, jButton7ActionPerformed, jButton8ActionPerformed, jButton9ActionPerformed Cómo funciona en conjunto: El código actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JPanel: paneles para agrupar y ordenar controles. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
/**
 *
 * @author alumno
 */
public class VentanaTablero extends javax.swing.JFrame {
    private boolean turno = true;
    /**
     * Creates new form VentanaTablero
     */
    public VentanaTablero() {
        initComponents();
        this.setSize(500,500);
        this.setLocationRelativeTo(null);
    }
    private void pulsarCasilla(javax.swing.JButton boton) {
        if (turno) {
            boton.setText("X");
            jLabel1.setText("Turno del Jugador O");
        } else {
            boton.setText("O");
            jLabel1.setText("Turno del Jugador X");
        }
        boton.setEnabled(false);
        turno = !turno;
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        pulsarCasilla(jButton1);
    }//GEN-LAST:event_jButton1ActionPerformed
    private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton2ActionPerformed
        pulsarCasilla(jButton2);
    }//GEN-LAST:event_jButton2ActionPerformed
    private void jButton4ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton4ActionPerformed
        pulsarCasilla(jButton4);
    }//GEN-LAST:event_jButton4ActionPerformed
    private void jButton3ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton3ActionPerformed
        pulsarCasilla(jButton3);
    }//GEN-LAST:event_jButton3ActionPerformed
    private void jButton5ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton5ActionPerformed
        pulsarCasilla(jButton5);
    }//GEN-LAST:event_jButton5ActionPerformed
    private void jButton7ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton7ActionPerformed
        pulsarCasilla(jButton7);
    }//GEN-LAST:event_jButton7ActionPerformed
    private void jButton6ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton6ActionPerformed
        pulsarCasilla(jButton6);
    }//GEN-LAST:event_jButton6ActionPerformed
    private void jButton8ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton8ActionPerformed
        pulsarCasilla(jButton8);
    }//GEN-LAST:event_jButton8ActionPerformed
    private void jButton9ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton9ActionPerformed
        pulsarCasilla(jButton9);
    }//GEN-LAST:event_jButton9ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JButton jButton2;
    private javax.swing.JButton jButton3;
```

```
    private javax.swing.JButton jButton4;
    private javax.swing.JButton jButton5;
    private javax.swing.JButton jButton6;
    private javax.swing.JButton jButton7;
    private javax.swing.JButton jButton8;
    private javax.swing.JButton jButton9;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JPanel jPanel1;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICAEXTRA · APARTADO03

Agenda con tabla: añadir y eliminar registros.

### VentanaAgenda.java — clase VentanaAgenda

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: DefaultTableModel, JButton, JFrame, JLabel, JOptionPane, JPanel, JScrollPane, JTable, JTextField Atributos detectados: private DefaultTableModel modelo Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); jButton2ActionPerformed(java.awt.event.ActionEvent evt); jTextField2ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed, jButton2ActionPerformed, jTextField2ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, añade una fila a una tabla, elimina una fila, actualiza texto visible, muestra un mensaje al usuario, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JPanel: paneles para agrupar y ordenar controles; JTable: tabla dentro de JScrollPane; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
/**
 *
 * @author alumno
 */
public class VentanaAgenda extends javax.swing.JFrame {
    private DefaultTableModel modelo;
    /**
     * Creates new form VentanaAgenda
     */
    public VentanaAgenda() {
        initComponents();
        this.setSize(500, 500);
        this.setLocationRelativeTo(null);
        modelo = new DefaultTableModel(new Object[] {"Nombre", "Telefono"}, 0);
        jTable1.setModel(modelo);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        String nombre = jTextField1.getText().trim();
        String telefono = jTextField2.getText().trim();
        if (nombre.isEmpty() || !telefono.matches("\\d{9}")) {
            javax.swing.JOptionPane.showMessageDialog(this, "El nombre no puede estar vacío y el teléfono
            debe tener 9 dígitos", "Advertencia", javax.swing.JOptionPane.WARNING_MESSAGE);
        } else {
            modelo.addRow(new Object[]{nombre, telefono});
            jLabel1.setText("Contador: " + modelo.getRowCount());
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton2ActionPerformed
        int fila = jTable1.getSelectedRow();
        if (fila == -1) {
            JOptionPane.showMessageDialog(this, "Por favor, selecciona una fila primero");
        } else {
            int confirmo = JOptionPane.showConfirmDialog(this, "¿Seguro que quieres eliminar este
            contacto?", "Confirmar", JOptionPane.YES_NO_OPTION);
            if (confirmo == JOptionPane.YES_OPTION) {
                modelo.removeRow(fila);
                jLabel1.setText("Contactos: " + modelo.getRowCount());
            }
        }
    }//GEN-LAST:event_jButton2ActionPerformed
    private void jTextField2ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jTextField2ActionPerformed
        // TODO add your handling code here:
    }//GEN-LAST:event_jTextField2ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JButton jButton2;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JPanel jPanel2;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTable jTable1;
    private javax.swing.JTextField jTextField1;
    private javax.swing.JTextField jTextField2;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICAEXTRA · APARTADO04

Registro de usuarios: contraseña, curso y aceptación de condiciones.

### VentanaUsuario.java — clase VentanaUsuario

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JCheckBox, JComboBox, JFrame, JLabel, JOptionPane, JPanel, JPasswordField, JTextField Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); botonRegistrarActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: botonRegistrarActionPerformed Cómo funciona en conjunto: El código lee texto introducido, lee un campo de contraseña, consulta una casilla u opción, actualiza texto visible, muestra un mensaje al usuario, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JCheckBox: casilla de verificación; JComboBox: lista desplegable con las opciones del ejercicio; JLabel: etiquetas para títulos e instrucciones; JPanel: paneles para agrupar y ordenar controles; JPasswordField: campos de contraseña; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

### Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
/**
 *
 * @author alumno
 */
public class VentanaUsuario extends javax.swing.JFrame {
    /**
     * Creates new form VentanaUsuario
     */
    public VentanaUsuario() {
        initComponents();
        this.setSize(500, 500);
        this.setLocationRelativeTo(null);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
```

# PRACTICAEXTRA · APARTADO05

Calculadora de propina: JSlider, porcentajes y actualización dinámica.

### VentanaPropina.java — clase VentanaPropina

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JCheckBox, JFrame, JLabel, JPanel, JSlider, JTextField Métodos detectados: calcularPropina(double importe, int porcentaje); actualizarTotal(); initComponents(); actionPerformed(java.awt.event.ActionEvent evt); stateChanged(javax.swing.event.ChangeEvent evt); actionPerformed(java.awt.event.ActionEvent evt); jCheckBox2ActionPerformed(java.awt.event.ActionEvent evt); jTextField1ActionPerformed(java.awt.event.ActionEvent evt); jSlider1StateChanged(javax.swing.event.ChangeEvent evt); main(String args[]); run() Eventos detectados: jCheckBox2ActionPerformed, jSlider1StateChanged, jTextField1ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, convierte texto a decimal, consulta una casilla u opción, actualiza texto visible, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JCheckBox: casilla de verificación; JLabel: etiquetas para títulos e instrucciones; JPanel: paneles para agrupar y ordenar controles; JSlider: deslizador con mínimo, máximo y valor inicial; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
/**
 *
 * @author alumno
 */
public class VentanaPropina extends javax.swing.JFrame {
    private double calcularPropina(double importe, int porcentaje) {
        return importe * (porcentaje / 100.0);
    }
    private void actualizarTotal() {
        try {
            double importe = Double.parseDouble(jTextField1.getText().trim());
            if (importe < 0) {
                throw new NumberFormatException();
            }
            int porcentaje = jSlider1.getValue();
            double propina = calcularPropina(importe, porcentaje);
            double total = importe + propina;
            if (jCheckBox2.isSelected()) {
                total = Math.ceil(total);
            }
            jLabel3.setText("Total: " + String.format("%.2f", total) + " €");
            jLabel3.setForeground(java.awt.Color.BLACK);
        } catch (NumberFormatException e) {
            jLabel3.setText("Importe no válido");
            jLabel3.setForeground(java.awt.Color.RED);
        }
    }
    /**
     * Creates new form VentanaPropina
     */
    public VentanaPropina() {
        initComponents();
        this.setSize(500, 500);
        this.setLocationRelativeTo(null);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jCheckBox2ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jCheckBox2ActionPerformed
        actualizarTotal();
    }//GEN-LAST:event_jCheckBox2ActionPerformed
    private void jTextField1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jTextField1ActionPerformed
        actualizarTotal();
    }//GEN-LAST:event_jTextField1ActionPerformed
    private void jSlider1StateChanged(javax.swing.event.ChangeEvent evt) {//GEN-
       FIRST:event_jSlider1StateChanged
        jLabel2.setText("Propina: " + jSlider1.getValue() + " %");
        actualizarTotal();
    }//GEN-LAST:event_jSlider1StateChanged
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JCheckBox jCheckBox2;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JLabel jLabel2;
    private javax.swing.JLabel jLabel3;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JPanel jPanel2;
    private javax.swing.JPanel jPanel3;
    private javax.swing.JSlider jSlider1;
    private javax.swing.JTextField jTextField1;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICAEXTRA · APARTADO06

Teclado PIN: botones numéricos, borrar y comprobación.

### VentanaPin.java — clase VentanaPin

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: JButton, JFrame, JOptionPane, JPanel, JPasswordField, JTextField Atributos detectados: private final String PIN_CORRECTO = "1234"; private int intentos = 0 Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); jBorrarActionPerformed(java.awt.event.ActionEvent evt); jOkActionPerformed(java.awt.event.ActionEvent evt); jNum1ActionPerformed(java.awt.event.ActionEvent evt) Eventos detectados: jBorrarActionPerformed, jNum1ActionPerformed, jOkActionPerformed Cómo funciona en conjunto: El código lee texto introducido, lee un campo de contraseña, actualiza texto visible, muestra un mensaje al usuario, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JPanel: paneles para agrupar y ordenar controles; JPasswordField: campos de contraseña; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo).

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
/**
 *
 * @author alumno
 */
public class VentanaPin extends javax.swing.JFrame {
    private final String PIN_CORRECTO = "1234";
    private int intentos = 0;
    /**
     * Creates new form VentanaPin
     */
    public VentanaPin() {
        initComponents();
        this.setSize(500, 500);
        this.setLocationRelativeTo(null);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jBorrarActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jBorrarActionPerformed
        String texto = new String(jPasswordField1.getPassword());
        if (texto.length() > 0) {
            jPasswordField1.setText(texto.substring(0, texto.length() - 1));
        }
    }//GEN-LAST:event_jBorrarActionPerformed
    private void jOkActionPerformed(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_jOkActionPerformed
        String pin = new String(jPasswordField1.getPassword());
        if (pin.length() < 4) {
            javax.swing.JOptionPane.showMessageDialog(this, "Introduce 4 dígitos");
            return;
        }
        if (pin.equals(PIN_CORRECTO)) {
            javax.swing.JOptionPane.showMessageDialog(this, "Acceso concedido");
            this.dispose();
        } else {
            intentos++;
            jPasswordField1.setText("");
            if (intentos >= 3) {
                javax.swing.JOptionPane.showMessageDialog(this, "Tarjeta bloqueada", "Error",
                javax.swing.JOptionPane.ERROR_MESSAGE);
                for (java.awt.Component c : jPanel1.getComponents()) {
                    c.setEnabled(false);
                }
            } else {
                javax.swing.JOptionPane.showMessageDialog(this, "PIN incorrecto. Intentos restantes: " +
                (3 - intentos));
            }
        }
    }//GEN-LAST:event_jOkActionPerformed
    private void jNum1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
      FIRST:event_jNum1ActionPerformed
       javax.swing.JButton btn = (javax.swing.JButton) evt.getSource();
       String textoActual = new String(jPasswordField1.getPassword());
       if (textoActual.length() < 4) {
           jPasswordField1.setText(textoActual + btn.getText());
       }
    }//GEN-LAST:event_jNum1ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jBorrar;
    private javax.swing.JButton jNum0;
    private javax.swing.JButton jNum1;
    private javax.swing.JButton jNum2;
    private javax.swing.JButton jNum3;
    private javax.swing.JButton jNum4;
    private javax.swing.JButton jNum5;
    private javax.swing.JButton jNum6;
    private javax.swing.JButton jNum7;
    private javax.swing.JButton jNum8;
```

```
    private javax.swing.JButton jNum9;
    private javax.swing.JButton jOk;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JPasswordField jPasswordField1;
    // End of variables declaration//GEN-END:variables
}
```

# PRACTICAEXTRA · APARTADO07

Lista de tareas: JList, DefaultListModel y menú.

## VentanaTareas.java — clase VentanaTareas

Ventana principal o pantalla de la aplicación. Hereda de JFrame y construye los controles en initComponents(). Componentes o clases Swing detectados: DefaultListModel, JButton, JFrame, JLabel, JList, JMenu, JMenuBar, JMenuItem, JOptionPane, JPanel, JScrollPane, JTextField Atributos detectados: private DefaultListModel<String> modelo Métodos detectados: initComponents(); actionPerformed(java.awt.event.ActionEvent evt); getSize(); getElementAt(int i); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); actionPerformed(java.awt.event.ActionEvent evt); jButton1ActionPerformed(java.awt.event.ActionEvent evt); jButton2ActionPerformed(java.awt.event.ActionEvent evt); jMenuItem1ActionPerformed(java.awt.event.ActionEvent evt); jMenuItem2ActionPerformed(java.awt.event.ActionEvent evt); main(String args[]); run() Eventos detectados: jButton1ActionPerformed, jButton2ActionPerformed, jMenuItem1ActionPerformed, jMenuItem2ActionPerformed Cómo funciona en conjunto: El código lee texto introducido, añade un elemento a una lista, elimina un elemento, actualiza texto visible, muestra un mensaje al usuario, muestra una ventana. Sigue el flujo desde el evento o constructor hasta el cambio de datos y la actualización de la ventana. En Design de NetBeans: crea el formulario correspondiente y arrastra JButton: botones para ejecutar acciones; JLabel: etiquetas para títulos e instrucciones; JList: lista dentro de JScrollPane; JMenuBar: barra de menú; JMenuItem: opciones dentro de un menú; JPanel: paneles para agrupar y ordenar controles; JTextField: campos de entrada de texto. Cambia la propiedad text para el texto visible y Change Variable Name para asignar nombres claros. Asocia los eventos desde la pestaña Events (icono del rayo). Para JList: crea DefaultListModel, asígnalo con setModel y modifica la lista con addElement, remove o clear. Comprueba que haya selección antes de eliminar.

## Código fuente completo del archivo:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/GUIForms/JFrame.java to edit this template
 */
package interfaz;
import javax.swing.DefaultListModel;
import javax.swing.JOptionPane;
/**
 *
 * @author alumno
 */
public class VentanaTareas extends javax.swing.JFrame {
    private DefaultListModel<String> modelo;
    /**
     * Creates new form VentanaTareas
     */
    public VentanaTareas() {
        initComponents();
        this.setSize(500, 500);
        this.setLocationRelativeTo(null);
        modelo = new DefaultListModel<>();
        jList1.setModel(modelo);
    }
    /**
     * This method is called from within the constructor to initialize the form.
     * WARNING: Do NOT modify this code. The content of this method is always
     * regenerated by the Form Editor.
     */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    // [Se omite el método initComponents() generado/auxiliar]
    private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton1ActionPerformed
        String tarea = jTextField1.getText().trim();
        if (tarea.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Tarea vacía", "Aviso", JOptionPane.WARNING_MESSAGE);
        } else if (modelo.contains(tarea)) {
            JOptionPane.showMessageDialog(this, "Tarea ya existe", "Aviso", JOptionPane.WARNING_MESSAGE);
        } else {
            modelo.addElement(tarea);
            jTextField1.setText("");
            jLabel1.setText("Pendientes: " + modelo.size());
        }
    }//GEN-LAST:event_jButton1ActionPerformed
    private void jButton2ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jButton2ActionPerformed
        int index = jList1.getSelectedIndex();
        if (index == -1) {
            JOptionPane.showMessageDialog(this, "Selecciona una tarea");
        } else {
            modelo.remove(index);
            jLabel1.setText("Pendientes: " + modelo.size());
        }
    }//GEN-LAST:event_jButton2ActionPerformed
    private void jMenuItem1ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jMenuItem1ActionPerformed
        int confirmar = JOptionPane.showConfirmDialog(this, "¿Vaciar lista?", "Confirmar",
        JOptionPane.YES_NO_OPTION);
        if (confirmar == JOptionPane.YES_OPTION) {
            modelo.clear();
            jLabel1.setText("Pendientes: 0");
        }
    }//GEN-LAST:event_jMenuItem1ActionPerformed
    private void jMenuItem2ActionPerformed(java.awt.event.ActionEvent evt) {//GEN-
    FIRST:event_jMenuItem2ActionPerformed
        System.exit(0);
    }//GEN-LAST:event_jMenuItem2ActionPerformed
    /**
     * @param args the command line arguments
     */
    // [Se omite el método main() generado/auxiliar]
    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton jButton1;
    private javax.swing.JButton jButton2;
    private javax.swing.JLabel jLabel1;
    private javax.swing.JList<String> jList1;
    private javax.swing.JMenu jMenu1;
    private javax.swing.JMenuBar jMenuBar1;
    private javax.swing.JMenuItem jMenuItem1;
    private javax.swing.JMenuItem jMenuItem2;
    private javax.swing.JPanel jPanel1;
    private javax.swing.JPanel jPanel2;
```

```
    private javax.swing.JPanel jPanel3;
    private javax.swing.JScrollPane jScrollPane1;
    private javax.swing.JTextField jTextField1;
    // End of variables declaration//GEN-END:variables
}
```

Guía general para ejecutar todos los proyectos Desde NetBeans: abre el proyecto, pulsa Clean and Build Project para compilar y luego Run Project (F6). Si hay varias ventanas, asegúrate de que la clase principal sea la que contiene public static void main. Para ejecutar un archivo concreto con main, usa Run File (Shift+F6). En el método main: el patrón habitual es crear la ventana dentro de

EventQueue.invokeLater y hacer setVisible(true). Si no se muestra nada, comprueba que se esté ejecutando la clase correcta y que el constructor llame a initComponents(). Depuración: mira Output, localiza el archivo y número de línea del error, coloca un breakpoint y ejecuta Debug. Comprueba los valores de las variables antes y después del evento. Comandos que debes reconocer getText() lee un campo; setText() cambia texto; getPassword() lee contraseña; isSelected() consulta casilla; Integer.parseInt() convierte a entero; Double.parseDouble() convierte a decimal; trim() elimina espacios exteriores; isEmpty() comprueba vacío; addRow() añade fila; removeRow() elimina fila; addElement() añade a lista; dispose() cierra un diálogo; setVisible(true) muestra una ventana. Fin del manual. Este documento contiene el código de los 44 archivos Java que se pudieron extraer del RAR.